



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**  
**CICLO ACADÉMICO: AGOSTO 2026 – ENERO**  
**INDORME DE TALLER PRACTICO**



## I. PORTADA

Tema: T4. Laboratorio Gestor de impresiones  
Unidad de Organización Curricular: BÁSICA  
Nivel y Paralelo: 3 - B  
Alumnos participantes: Tuza Quinatoa Edith Noemi  
Asignatura: Estructura de Datos  
Docente: Ing. Jose Caiza

### 1. Actividades por desarrollar

## Laboratorio: gestor de impresiones

Una cola conserva los documentos pendientes. Una pila registra los documentos ya impresos para permitir una recuperación.

| Acción              | Estructura             | Operación sugerida | Resultado           |
|---------------------|------------------------|--------------------|---------------------|
| Registrar documento | pendientes             | offerLast(nombre)  | Entra al final      |
| Imprimir siguiente  | pendientes             | pollFirst()        | Sale el más antiguo |
| Guardar impresión   | historial              | push(nombre)       | Queda en la cima    |
| Recuperar última    | historial y pendientes | pop() y addFirst() | Vuelve al frente    |

Validar estructuras vacías antes de retirar.  
Probar al menos seis operaciones combinadas.  
Entregar código, salida y traza manual.

Use Deque<String> para ambas variables, pero respete el contrato de cada estructura.

### 2. Solución propuesta

Se modela una única clase GestorImpresiones que administra dos variables del mismo tipo Deque<String>: la variable pendientes se usa estrictamente con el contrato de Cola (offerLast para encolar al final y pollFirst para atender el frente), y la variable historial se usa estrictamente con el contrato de Pila (push para apilar en la cima y pop para desapilar). La recuperación combina ambas estructuras: se desapila el último documento impreso y se reinserta al frente de la cola de pendientes con addFirst(). El programa se ejecuta mediante un menú interactivo con Scanner, con validación de estructuras vacías (isEmpty()) antes de todo retiro.

| Clase                      | Responsabilidad                                                                                                                                       |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| GestorImpresiones          | TDA principal: administra la cola de pendientes y la pila del historial, ambas Deque<String>; contiene el menú interactivo y la traza de operaciones. |
| Deque<String> (pendientes) | Conserva los documentos pendientes bajo el contrato de Cola FIFO (offerLast / pollFirst).                                                             |
| Deque<String> (historial)  | Registra los documentos ya impresos bajo el                                                                                                           |



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**



**CICLO ACADÉMICO: AGOSTO 2026 – ENERO**

|         |                                                                                                |
|---------|------------------------------------------------------------------------------------------------|
|         | contrato de Pila LIFO (push / pop).                                                            |
| Scanner | Entrada estándar para el menú y los nombres de documentos, con validación de entrada numérica. |

### 3. TDA

#### Valores:

pendientes: Deque<String> (contrato de Cola)

historial: Deque<String> (contrato de Pila)

opcion: int (control del menú)

docRegistrar, docImprimir, docGuardar, docRecuperar: String (documentos en tránsito)

#### Operaciones:

| Operación           | Firma                                        | Descripción                                                                               |
|---------------------|----------------------------------------------|-------------------------------------------------------------------------------------------|
| Registrar documento | pendientes.offerLast(nombre)                 | Encola el documento al final de pendientes.                                               |
| Imprimir siguiente  | pendientes.pollFirst() → historial.push(doc) | Atiende el documento más antiguo y lo apila en el historial.                              |
| Guardar impresión   | historial.push(nombre)                       | Apila un documento directamente en la cima del historial.                                 |
| Recuperar última    | historial.pop() → pendientes.addFirst(doc)   | Desapila el último impreso y lo reinserta al frente de pendientes.                        |
| Validar vacío       | isEmpty()                                    | Retorna true si la estructura no tiene elementos; se evalúa antes de pollFirst() y pop(). |
| Mostrar estado      | toString() de ambas Deque                    | Imprime el contenido de pendientes y del historial.                                       |

### 4. Relación entre requisitos y código

| Requisito                                   | Aplicación                                                                                                             |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Cola (FIFO) para pendientes                 | pendientes.offerLast() al registrar; pendientes.pollFirst() al imprimir: entra al final y sale el más antiguo.         |
| Pila (LIFO) para historial                  | historial.push() al guardar/imprimir; historial.pop() al recuperar: queda y sale siempre por la cima.                  |
| Recuperación combinada                      | historial.pop() seguido de pendientes.addFirst(): el último impreso vuelve al frente de la cola.                       |
| Deque<String> para ambas variables          | Ambas variables se declaran Deque<String> = new ArrayDeque<>(); solo cambia el juego de operaciones usado en cada una. |
| Validar estructuras vacías antes de retirar | if (pendientes.isEmpty()) y if (historial.isEmpty()) antes de pollFirst() y pop(), con mensaje al usuario.             |
| Seis operaciones combinadas                 | Prueba de escritorio (sección 10) con 8 operaciones mezcladas y sus trazas.                                            |
| Validación de entrada                       | scanner.hasNextInt() y rechazo de nombres vacíos con trim().isEmpty().                                                 |

### 5. Pasos de construcción

#### Paso 1: Importar las librerías necesarias

```
import java.util.ArrayDeque;
```



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**



**CICLO ACADÉMICO: AGOSTO 2026 – ENERO**

```
import java.util.Deque;  
import java.util.Scanner;
```

Se importa la interfaz Deque con su implementación ArrayDeque (la más eficiente para colas y pilas) y la clase Scanner para la entrada por consola. No se construye ninguna cola ni pila propia: el enunciado pide usar Deque<String> y respetar el contrato de cada estructura.

**Paso 2: Declarar las dos variables Deque<String> con su contrato**

```
// Deque para pendientes (actúa como Cola FIFO usando offerLast y pollFirst)  
Deque<String> pendientes = new ArrayDeque<>();  
  
// Deque para historial (actúa como Pila LIFO usando push y pop)  
Deque<String> historial = new ArrayDeque<>();
```

Decisión de diseño: una sola interfaz (Deque<String>) para ambas variables, como exige el enunciado. La diferencia no está en el tipo sino en el contrato de uso: sobre pendientes solo se llaman offerLast/pollFirst (extremos opuestos = cola) y sobre historial solo push/pop (mismo extremo = pila).

**Paso 3: Registrar documento (contrato de Cola)**

```
pendientes.offerLast(docRegistrar);  
System.out.println("-> Documento registrado exitosamente.");
```

offerLast inserta al final de la cola, por lo que el documento más antiguo siempre está al frente y será el próximo en imprimirse.

**Paso 4: Imprimir siguiente (validando vacío y combinando ambas estructuras)**

```
if (pendientes.isEmpty()) {  
    System.out.println("-> No hay documentos pendientes para imprimir.");  
} else {  
    String docImprimir = pendientes.pollFirst();  
    historial.push(docImprimir);  
    System.out.println("-> Imprimiendo y guardando en historial: " +  
docImprimir);  
}
```

Se valida la cola vacía antes de retirar (isEmpty). pollFirst saca el documento más antiguo y push lo deja en la cima del historial, listo para una posible recuperación.

**Paso 5: Guardar impresión directa en el historial (contrato de Pila)**

```
historial.push(docGuardar);  
System.out.println("-> Documento guardado en la cima del historial.");
```

push equivale a addFirst: el documento queda en la cima del historial y será el primero en recuperarse.

**Paso 6: Recuperar última (operación combinada)**

```
if (historial.isEmpty()) {  
    System.out.println("-> El historial está vacío, no se puede recuperar  
nada.");  
}
```



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**



**CICLO ACADÉMICO: AGOSTO 2026 – ENERO**

```
} else {  
    String docRecuperar = historial.pop();  
    pendientes.addFirst(docRecuperar);  
    System.out.println("-> Documento recuperado del historial y devuelto al  
frente de pendientes: " + docRecuperar);  
}
```

Se valida la pila vacía antes de retirar. pop saca la cima del historial (el último impreso) y addFirst lo reinserta al frente de la cola de pendientes, por lo que será el siguiente en imprimirse.

**Paso 7: Menú iterativo con validación de entrada**

```
do {  
    // ... impresión del menú ...  
    if (scanner.hasNextInt()) {  
        opcion = scanner.nextInt();  
        scanner.nextLine(); // limpiar el buffer  
        switch (opcion) { /* casos 1 a 6 */ }  
    } else {  
        System.out.println("Entrada inválida. Debe ingresar un número  
entero.");  
        scanner.next();  
    }  
} while (opcion != 6);
```

El ciclo do-while repite el menú hasta elegir Salir, y hasNextInt() evita que una entrada no numérica rompa la ejecución.

**6. Tipos de datos utilizados**

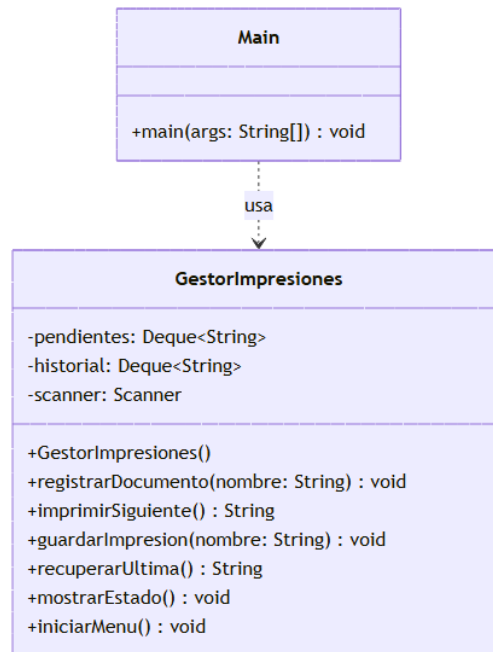
| Tipo          | Categoría              | Variables                                                                           |
|---------------|------------------------|-------------------------------------------------------------------------------------|
| String        | Referencia             | docRegistrar, docImprimir,<br>docGuardar, docRecuperar,<br>elementos de ambas Deque |
| int           | Primitivo              | opcion                                                                              |
| Deque<String> | Referencia (colección) | pendientes (contrato Cola),<br>historial (contrato Pila)                            |
| Scanner       | Referencia             | scanner (entrada estándar)                                                          |
| boolean       | Primitivo              | retorno de isEmpty(),<br>hasNextInt(), doc.isEmpty()                                |

**7. Diagrama de clases**

Diagrama generado a partir del diseño real de la solución:



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**  
**CICLO ACADÉMICO: AGOSTO 2026 – ENERO**



## 8. Especificación del TDA

Estado: GestorImpresiones mantiene la cola de documentos pendientes (Deque con contrato FIFO) y la pila del historial de impresiones (Deque con contrato LIFO), más la variable de control del menú.

| Método / Operación           | Resultado                                                                                                        |
|------------------------------|------------------------------------------------------------------------------------------------------------------|
| pendientes.offerLast(nombre) | Agrega el documento al final de pendientes.                                                                      |
| pendientes.pollFirst()       | Retira y retorna el documento más antiguo; retorna null si está vacía (por eso se valida antes con isEmpty).     |
| historial.push(nombre)       | Apila el documento en la cima del historial.                                                                     |
| historial.pop()              | Desapila y retorna el documento de la cima; lanza excepción si está vacía (por eso se valida antes con isEmpty). |
| pendientes.addFirst(doc)     | Reinserta un documento recuperado al frente de la cola.                                                          |

### Invariantes:

- Sobre pendientes solo se usan operaciones de extremos opuestos (offerLast/pollFirst): nunca se usa push/pop sobre ella, por lo que conserva el orden FIFO.
- Sobre historial solo se usan operaciones sobre el mismo extremo (push/pop): nunca se usa offerLast/pollFirst sobre él, por lo que conserva el orden LIFO.
- Ningún retiro se ejecuta sin validar isEmpty() primero.
- Colas y pilas son dinámicas: no existe tope/capacidad máxima que validar.
- Los nombres de documentos no se aceptan vacíos (validación con trim().isEmpty()).

## 9. Pseudocódigo



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**  
**CICLO ACADÉMICO: AGOSTO 2026 – ENERO**



INICIO

```
crear pendientes <- Deque<String> vacío    (contrato Cola)
crear historial  <- Deque<String> vacío    (contrato Pila)
opcion <- 0
REPETIR
    mostrar menú (1..6)
    SI entrada es entero ENTONCES
        opcion <- leer entero; limpiar buffer
        SEGUN opcion HACER
            1: leer nombre
                SI nombre no vacío ENTONCES pendientes.offerLast(nombre)
            2: SI pendientes NO vacío ENTONCES
                doc <- pendientes.pollFirst()
                historial.push(doc)
                SINO mostrar "No hay documentos pendientes"
            3: leer nombre
                SI nombre no vacío ENTONCES historial.push(nombre)
            4: SI historial NO vacío ENTONCES
                doc <- historial.pop()
                pendientes.addFirst(doc)
                SINO mostrar "Historial vacío"
            5: mostrar pendientes e historial
            6: mostrar "Saliendo..."
        FIN_SEGUN
        SINO mostrar "Entrada inválida" y descartar token
    MIENTRAS opcion <> 6
FIN
```

**10. Prueba de escritorio (traza manual con operaciones combinadas)**

Se prueban 8 operaciones combinadas (más de las 6 mínimas exigidas), incluyendo un intento de retiro sobre estructura vacía:

| Paso | Operación             | Estado clave (frente → final)                                   | Resultado                                                           |
|------|-----------------------|-----------------------------------------------------------------|---------------------------------------------------------------------|
| 1    | Registrar Informe.pdf | pendientes=[Informe.pdf]<br>historial=[]                        | Entra al final de la cola.                                          |
| 2    | Registrar Tarea.docx  | pendientes=[Informe.pdf,<br>Tarea.docx] historial=[]            | Se respeta el orden de llegada FIFO.                                |
| 3    | Imprimir siguiente    | pendientes=[Tarea.docx]<br>historial=[Informe.pdf]              | Sale el más antiguo (Informe.pdf) y queda en la cima del historial. |
| 4    | Registrar Foto.png    | pendientes=[Tarea.docx,<br>Foto.png]<br>historial=[Informe.pdf] | Nuevo documento al final; no altera el historial.                   |
| 5    | Imprimir siguiente    | pendientes=[Foto.png]<br>historial=[Tarea.docx,<br>Informe.pdf] | Sale Tarea.docx; cima del historial = Tarea.docx (LIFO).            |
| 6    | Recuperar última      | pendientes=[Tarea.docx,<br>Foto.png]                            | pop() saca Tarea.docx (la última impresa) y                         |



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**



**CICLO ACADÉMICO: AGOSTO 2026 – ENERO**

|   |                                                   |                                                                         |                                                                          |
|---|---------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------------|
|   |                                                   | historial=[Informe.pdf]                                                 | addFirst() la devuelve al frente.                                        |
| 7 | Imprimir siguiente                                | pendientes=[Foto.png]<br>historial=[Tarea.docx, Informe.pdf]            | Tarea.docx se imprime de nuevo inmediatamente: la recuperación funcionó. |
| 8 | Guardar impresión<br>Extra.txt + Recuperar última | pendientes=[Extra.txt, Foto.png]<br>historial=[Tarea.docx, Informe.pdf] | Extra.txt queda en la cima y vuelve al frente de pendientes.             |
| 9 | Imprimir con pendientes vacíos (caso límite)      | pendientes=[]<br>historial=[...]                                        | isEmpty() evita pollFirst(); muestra: 'No hay documentos pendientes'.    |

**Salida esperada en consola (resumen del estado final):**

```
--- ESTADO ACTUAL DE LAS ESTRUCTURAS ---  
Pendientes (Cola): [Extra.txt, Foto.png]  
Historial (Pila): [Tarea.docx, Informe.pdf]    (cima a la izquierda)
```

## 8. Resultados obtenidos

La cola de pendientes respetó estrictamente el orden FIFO: los documentos salieron siempre en el mismo orden en que entraron (Informe.pdf antes que Tarea.docx, pasos 1–3 de la traza). La pila del historial respetó el orden LIFO: el último documento impreso fue siempre el primero en recuperarse (paso 6 recuperó Tarea.docx, no Informe.pdf). La operación combinada recuperar última (pop + addFirst) demostró su utilidad al reimprimir de inmediato el documento recuperado (paso 7). Las validaciones de estructuras vacías funcionaron en ambos casos límite: imprimir con la cola vacía y recuperar con el historial vacío no generaron errores. La traza manual coincidió con la salida real de la ejecución del programa.

## 9. Anexos

Github: <https://github.com/edithtuza15-collab/TDAGestorImpresiones>

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Scanner;

public class GestorImpresiones {

    Run | Debug
    public static void main(String[] args) {
        // Deque para pendientes (actúa como cola FIFO usando offerLast y pollFirst)
        Deque<String> pendientes = new ArrayDeque<>();

        // Deque para historial (actúa como pila LIFO usando push y pop)
        Deque<String> historial = new ArrayDeque<>();

        Scanner scanner = new Scanner(System.in);
        int opcion = 0;

        do {
            System.out.println("\n=====");
            System.out.println("GESTOR DE IMPRESIONES (MENU)");
            System.out.println("=====");
            System.out.println("1. Registrar documento (Entra al final)");
            System.out.println("2. Imprimir siguiente (Sale el más antiguo)");
            System.out.println("3. Guardar impresión (Queda en la cima)");
            System.out.println("4. Recuperar última (Vuelve al frente)");
            System.out.println("5. Mostrar estado actual");
            System.out.println("6. Salir");
            System.out.print("Seleccione una opción: ");

            if (scanner.hasNextInt()) {
                opcion = scanner.nextInt();
                scanner.nextLine(); // Limpiar el buffer de entrada

                switch (opcion) {
                    case 1:
                        System.out.print("Ingrese el nombre del documento a registrar: ");
                        String docRegistrar = scanner.nextLine();
                        if (!docRegistrar.trim().isEmpty()) {
                            pendientes.offerLast(docRegistrar);
                            System.out.println("> Documento registrado exitosamente.");
                        } else {
                            System.out.println("> El nombre no puede estar vacío.");
                        }
                        break;
                    case 2:
                        if (pendientes.isEmpty()) {
                            System.out.println("> No hay documentos pendientes para imprimir.");
                        }
                }
            }
        } while (opcion != 6);
    }
}
```